《算法设计与分析》实验报告

实验一　递归与分治策略

报告书			
姓名	顾凯琪	指导教师	方贤忠
学号	226001624	日　　期	2025/3/14
班级	22 级计算机 2 班		

实验内容

[要求先在 OJ 上提交通过]

1、[OJ]递推/递归与分治策略 1

2、[OJ]递推/递归与分治策略 2

3、[OJ]递推/递归与分治策略-堂练

实验目的

1） 理解递推、递归与分治策略的基本原理

2） 通过实验，掌握递推、递归与分治策略的核心思想及其适用场景。

3） 理解递推与递归的关系，以及分治策略如何通过分解问题降低复杂度。

4） 学习递推策略的循环实现方式，掌握其在动态规划、数列计算等问题中的应用。

5） 学习掌握本类算法的设计与实现，在 OJ 上提交算法代码并测试通过，完成实验报告书写。

【如有查重雷同，请在此说明】

代码独立完成，无雷同。

【如有 OJ 未完成，请在此说明，并在本实验文档中补全】

自行完善：

OJ 实验 1 作业正常时间完成。

OJ 实验 2 作业正常时间完成。

OJ 实验 3 堂练全部完成。

实验过程和步骤

【第一部分】

Module 1: 递推/递归与分治策略 1

【1、 王老师爬楼梯（1-1000）】

1000 Problem A

时间限制：2000/1000 MS(Java/Others) | 内存限制：65536/32768 KB(Java/Others)

Submits：80 | Solved：0

题目描述

王老师爬楼梯，他可以每次走1级或者2级或者3级楼梯，输入楼梯的级数，求不同的走法数。（要求递推求解）如果N很大，需要高精度计算。

输入要求

一个整数N，N<=1000。

输出要求

共有多少种走法。

【2、程序代码及注释】

/*

王老师走台阶，依次可以走一节、两节、或者三阶楼梯

首先自顶向下分析，如果去掉高精度,就是 f[n]=f[n-1]+f[n-2]+f[n-3],

其意义为走 n 个台阶时的步骤种类 = 最后走 1 个台阶 + 最后走 2 个台阶 + 最后走 3 个台阶

接着我们再自底向上分析，得到走一阶，两阶，三阶的走法分别为 1 次、2 次、4 次

接着使用一个二维数组，每个二维数组的第二维存储的是每个数的 5 位数据，防止数据溢出的问题，相当于每行存储了一个数。

*/

```
1. #include<iostream>
2. using namespace std;
3.
4. int a[1000][100] = {0}; // 定义一个二维数组 a，用于存储数列的每一位，每列存储 5 位数字
5.
6. // 函数 fun 用于计算数列的第 n 项
7. void fun(int n) {
8.     int i, q, j;
9.     a[0][99] = 1; // 第 0 项的最低位是 1
```

```
    a[1][99] = 2; // 第1项的最低位是2
    a[2][99] = 4; // 第2项的最低位是4

    // 从第3项开始计算
    for (i = 3; i < n; i++) {
        // 计算当前项的最低位
        j = a[i - 3][99] + a[i - 2][99] + a[i - 1][99];
        a[i][99] = j % 100000; // 取后5位存储到a[i][99]

        // 从倒数第二列开始，逐列计算高位
        for (q = 98; q >= 0; q--) {
            // 计算当前列的值，并加上来自低位的进位
            j = a[i - 3][q] + a[i - 2][q] + a[i - 1][q] + j / 100000;
            a[i][q] = j % 100000; // 取后5位存储到a[i][q]
        }
    }
}

int main() {
    int n;
    cin >> n; // 输入n，表示要计算数列的第n项

    int j, s = 0;
    fun(n); // 调用fun函数计算数列的第n项

    // 循环输出数列的第n-1项
    for (int j = 0; j < 100; j++) {
        if (a[n - 1][j] != 0 && s == 0) {
            // 第一个非零数字，不补零
            cout << a[n - 1][j];
            s = 1; // 标记已经输出了第一个非零数字
        } else if (s == 1) {
            // 后续数字，固定输出5位，不足5位的前面补零
            printf("%05d", a[n - 1][j]);
        }
    }
    cout << endl; // 输出换行
    return 0;
}
```

【3、遇到的问题及解决方法】

1.Time Limit Exceed,使用递归方法进行计算，中间产生多次计算问题，事件复杂度过高。

Run Time Error，直接使用 unsigned long long 无法解决高精度问题.

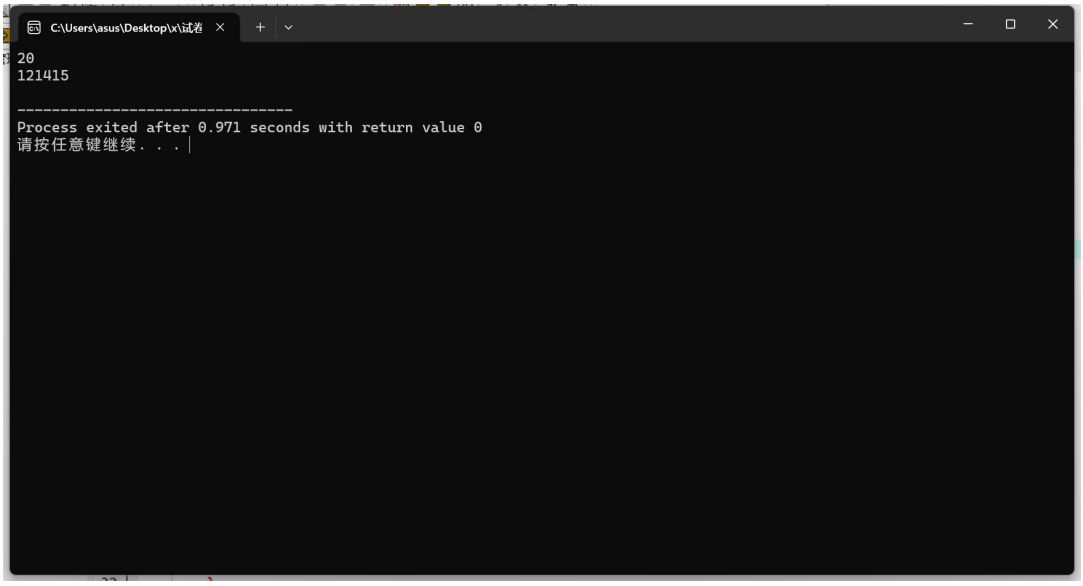
2.整型数据溢出，题目要求输入的数据所产生的数据过大，产生了溢出，需要使用二维数组，将每次计算所产生的低位和进位记录，并最终整合成一个大数组，每行实现一个大数。

【4、算法分析与知识点】

本实验使用了动态规划来解决问题，首先我们将原问题分解成若干子问题，得到计算公式，接着再初始化边界条件，得到台阶数分别为 1、2、3 时的不同方法，最后使用递推计算得到结果。

【5、AC 截图及运行测试截图】

5575449	226001624	1000	Accepted	--- KB	--- MS	G++	--- B	2025-02-24 17:48:35



【1、 铺砖（1-1001）】



【2、程序代码及注释】

```
/*
    这题与上一题一样是一个动态规划的大数问题，我们只要将其正确地分割为若干子问题即可
```

f[n] = f[n-1] + 2*f[n-2]

如果考虑最后一列由 1*2 方块来竖着放，这样就是 f[n-1],
接着我们考虑最后 2*2 的地方，有三种方式：一个 2*2,二个 1*2 竖着,二个 1*2 横着,
但是两个 1*2 竖着放置与第一部分的放置方法重复，所以我们得到第二个子问题为 2*f[n-2]。 */

```
#include<iostream>
using namespace std;

int a[1000][100] = {0}; // 定义一个二维数组 a，用于存储数列的每一位，每列存储 5 位数字

// 函数 fun 用于计算数列的第 n 项
void fun(int n) {
    int i, q, j;
    a[0][99] = 1; // 第 0 项的最低位是 1
    a[1][99] = 3; // 第 1 项的最低位是 3

    // 从第 2 项开始计算
    for (i = 2; i < n; i++) {
        // 计算当前项的最低位
        j = a[i - 1][99] + 2 * a[i - 2][99];
        a[i][99] = j % 100000; // 取后 5 位存储到 a[i][99]

        // 从倒数第二列开始，逐列计算高位
        for (q = 98; q >= 0; q--) {
            // 计算当前列的值，并加上来自低位的进位
            j = a[i - 1][q] + 2 * a[i - 2][q] + j / 100000;
            a[i][q] = j % 100000; // 取后 5 位存储到 a[i][q]
        }
    }
}

int main() {
    int n;
    cin >> n; // 输入 n，表示要计算数列的第 n 项

    int j, s = 0;
    fun(n); // 调用 fun 函数计算数列的第 n 项

    // 循环输出数列的第 n-1 项
```

```
    for (int j = 0; j < 100; j++) {
        if (a[n - 1][j] != 0 && s == 0) {
            // 第一个非零数字，不补零
            cout << a[n - 1][j];
            s = 1; // 标记已经输出了第一个非零数字
        } else if (s == 1) {
            // 后续数字，固定输出 5 位，不足 5 位的前面补零
            printf("%05d", a[n - 1][j]);
        }
    }
    cout << endl; // 输出换行
    return 0;
}
```

【3、遇到的问题及解决方法】

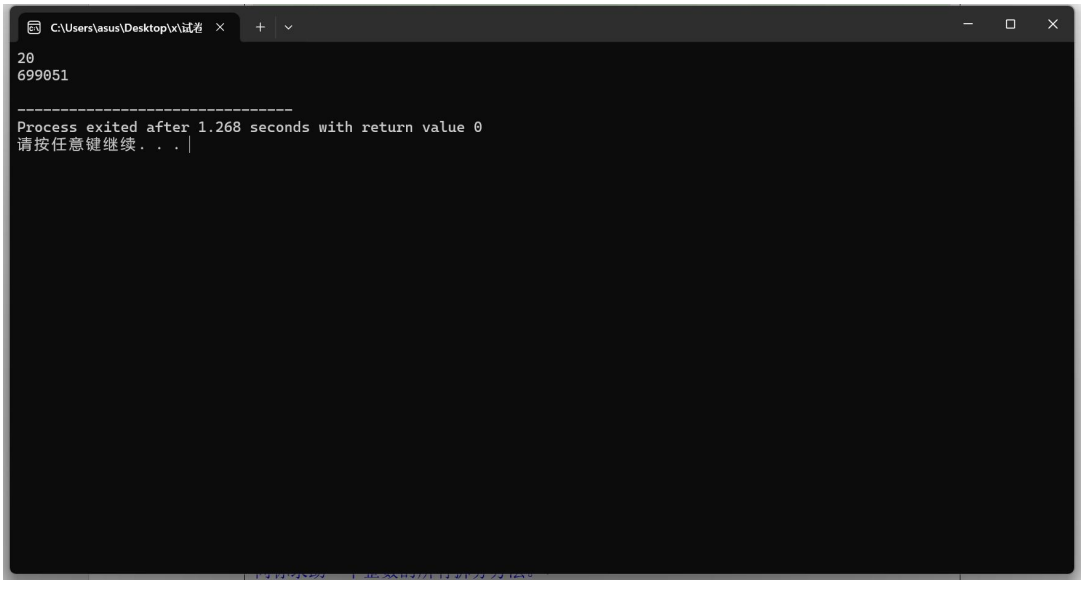
Compile Error，'long' followed by 'long' is illegal,想直接使用 C++中自带的数据类型解决，但还是不够用，所以采用数组。

【4、算法分析与知识点】

本实验使用了动态规划来解决问题，首先我们将原问题分解成若干子问题，得到计算公式 f[n]=f[n-1]+2*f[n-2]，接着再初始化边界条件，得到铺砖列数分别为 1、2、3 时的不同方法为 1，3，5，最后使用递推计算得到结果。

【5、AC 截图及运行测试截图】

5575451	226001624	1001	Accepted	--- KB	--- MS	G++	--- B	2025-02-24 17:50:15



【1、 过河卒（1-1002）】

3018 dt7-过河卒/noip2002

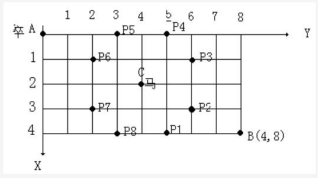
时间限制：2000/1000 MS(Java/Others) | 内存限制：65536/32768 KB(Java/Others)

Submits：308 | Solved：97

题目描述

棋盘上A点有一个过河卒，需要走到目标B点。卒行走的规则：可以向下或向右。同时在棋盘上的任一点有一个对方的马(如图中的C点)，该马所在的点和所有跳跃一步可达的点称为对方马的控制点。例如图中C点上的马可以控制9个点。卒不能走到对方马的控制点。

棋盘用坐标表示，A点坐标(0,0)、B点坐标(n, m) (n,m为不超过20的整数,并由键盘输入),同样马的位置坐标C是需要给出的(C≠A,且C≠B)。现在要求你计算出卒从A点能够到达B点的路径条数。



【2、程序代码及注释】

/*

左上角位置为（0，0）的卒想要到位置为（n，m）的目的地 B，其中存在一个马，在马的攻击范围点，卒不能通行。

输入 B 点的坐标(n, m)以及对方马的坐标（ x ， y ）。

代码采用动态规划思想，递推计算从 (0,0) 到 (n,m) 的路径数，同时避开马的攻击范围。

将问题拆解为子问题，设 mp[i][j] 为到达 (i,j) 的路径数，起点 mp[0][0] = 1。第一行和第一列的路径数由前一格继承，若受马控制则设为 0。

递推关系为 mp[i][j] = mp[i-1][j] + mp[i][j-1]，仅当 (i,j) 不受马攻击时成立。最终，mp[n][m] 即为符合条件的路径数。

*/

```
#include<bits/stdc++.h>
using namespace std;

// 定义一个二维数组用于存储动态规划的状态
long long  mp[20][20];

// check 函数用于判断给定位置(i,j)是否是“马点”（即马的攻击范围）。
// 如果当前位置在(x, y)附近（即马的攻击范围），返回1，否则返回0。
int check(int i, int j, int x, int y){
    // 判断位置(i, j)是否满足马的攻击范围：马的攻击范围是横向或纵向间隔2格，另一个方向间隔1格
    return (abs(i-x) + abs(j-y) == 3 && i != x && j != y) || (i == x && j == y);
}

// solve 函数用来计算从(0,0)到(n, m)的路径数，路径中不可经过马的攻击
```

```
    范围(x, y)。
// n 是行数，m 是列数，x 是马的起始 x 坐标，y 是马的起始 y 坐标。
long long solve(int n, int m, int x, int y){
    // 初始化起点(0, 0)的路径数为1
    mp[0][0] = 1;

    // 初始化第一列路径数
    for(int i = 1; i <= n; ++i){
        // 如果当前位置是马的攻击点，则不能经过，路径数为0；否则路径数等于上一行路径数
        mp[i][0] = (check(i, 0, x, y)? 0 : mp[i-1][0]);
    }

    // 初始化第一行路径数
    for(int j = 1; j <= m; ++j){
        // 如果当前位置是马的攻击点，则不能经过，路径数为0；否则路径数等于上一列路径数
        mp[0][j] = (check(0,j,x,y)? 0 : mp[0][j-1]);
    }

    // 计算其他位置的路径数，动态规划转移方程
    for(int i = 1; i <= n; ++i){
        for(int j = 1; j <= m; ++j){
            // 如果当前位置是马的攻击点，则路径数为0；否则路径数为从上方和左方的路径数之和
            mp[i][j] = (check(i,j,x,y)? 0 : mp[i-1][j] + mp[i][j-1]);
        }
    }

    // 返回终点(n, m)的路径数
    return mp[n][m];
}

int main(){
    int n, m, x, y;
    // 输入行数n、列数m，马的位置(x, y)
    cin >> n >> m >> x >> y;

    // 调用solve函数计算并输出从(0, 0)到(n, m)的路径数
    cout << solve(n, m, x, y) << endl;

    return 0;
```

```
53.}
```

【3、遇到的问题及解决方法】

标准 C++库中，size 是保留字，使用 size 会产生[Error] reference to 'size' is ambiguous 的报错，需要将 size 换成自定义标识符，例如 size1。

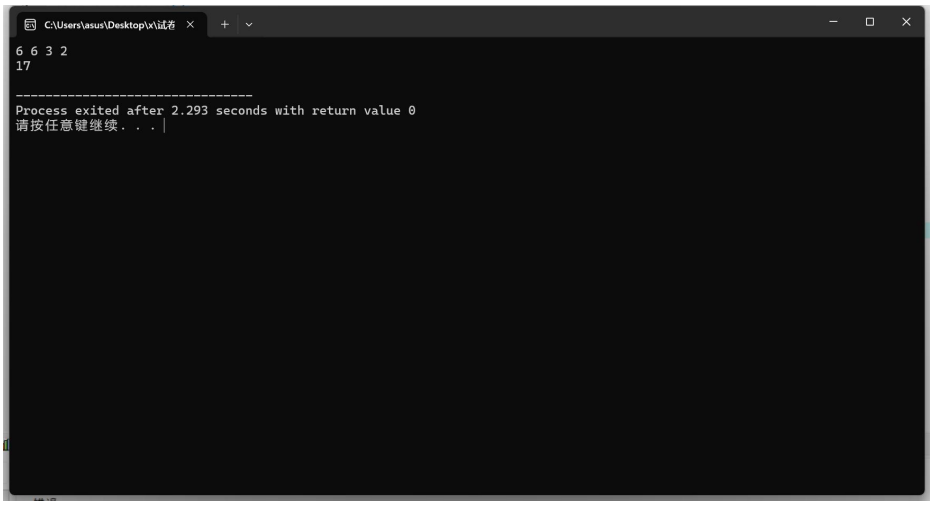
【4、算法分析与知识点】

本实验利用动态规划的思想计算从 (0,0) 到 (n,m) 的路径数，同时避开“马的攻击范围”。mp[i][j] 记录走到 (i,j) 的路径数，起点初始化为 1。第一行和第一列路径数由前一格继承，若被马攻击则设为 0。随后遍历整个网格，对每个非受限格子 (i,j)，其路径数为 mp[i-1][j] + mp[i][j-1]，即从上方和左方来的路径总和。最终，mp[n][m] 得出符合条件的路径数。

核心思想是动态规划递推，避开受限区域，逐步累积可行路径。

【5、AC 截图及运行测试截图】

5575455	226001624	1002	Accepted	--- KB	--- MS	G++	--- B	2025-02-24 17:53:44



Module 2: 递推/递归与分治策略 2

【1、 第 k 小的数（2-1000）】



1000 Problem A

时间限制：2000/1000 MS(Java/Others) | 内存限制：65536/32768 KB(Java/Others)

Submits：75 | Solved：60

题目描述

输入n个数，求其中第k小的数。（要求采用分治法完成，不建议采用完整的排序）

输入要求

第一行包含两个整数n和k；n<1000，1<=K<=n

第二行包含n个整数。

输出要求

输出第k小的那个整数。

【2、程序代码及注释】

/*输入 n 个数，求其中第 k 小的数。（要求采用分治法完成，不建议采用完整的排序）
第一行包含两个整数 n 和 k；n<1000，1<=K<=n
第二行包含 n 个整数。

可以基于快速选择算法，利用快速排序的分治策略，在不完全排序的情况下直接找到第 k 小的元素，提高效率。

Solve 通过双指针划分，选取 a[s] 作为基准值，使其左侧元素小于基准，右侧元素大于基准。然后根据 k-1 与基准索引 i 的比较，递归至左或右部分，确保每次递归减少搜索空间，使得平均时间复杂度为 O(n)。

main 负责输入数组和 k，调用 Solve 进行查找并输出结果，最终在 O(n) 时间内找到第 k 小的元素。*/

```
#include <iostream>
using namespace std;
#define MAXN 1000  // 定义数组最大长度

// 使用快速选择算法查找数组中第 k 小的元素
int Solve(int a[], int s, int t, int k) {
    int i = s, j = t;
    int temp = a[s];  // 选取基准值（通常是区间的第一个元素）

    // 快速划分
    while (i < j) {
        while (i < j && a[j] >= temp) j--;  // 从右向左寻找比基准小的元素
        a[i] = a[j];  // 将小的元素放到左侧

        while (i < j && a[i] <= temp) i++;  // 从左向右寻找比基准大的元素
        a[j] = a[i];  // 将大的元素放到右侧
    }
```

```
    a[i] = temp;  // 基准值归位

    // 判断第 k 小的元素是否正好是当前的划分点
    if (k - 1 == i) return a[i];

    // 递归到左半部分或右半部分继续查找
    return (k - 1 < i) ? Solve(a, s, i - 1, k) : Solve(a, i + 1
, t, k);
}

int main() {
    int n, k;
    int a[MAXN];

    // 输入数组大小 n 和要查找的第 k 小元素
    cin >> n >> k;

    // 输入数组元素
    for (int i = 0; i < n; i++) cin >> a[i];

    // 调用 Solve 函数求解并输出结果
    cout << Solve(a, 0, n - 1, k) << endl;

    return 0;
}
```

【3、遇到的问题及解决方法】

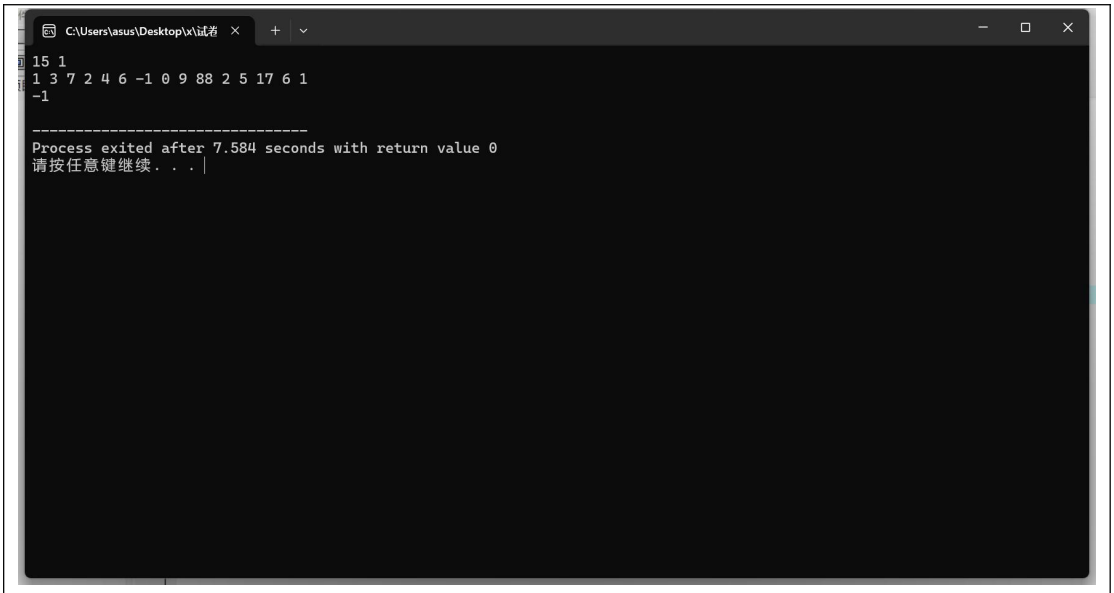
本题我最开始使用的算法是使用快排将整个数组进行排序，但是其实我们只要找到第 k 个小的数就行了，无需完整的排序，因此采用分治的思想，每次递归排序完，可以直接与 k 进行比较，时间复杂度可以退化根号次。

【4、算法分析与知识点】

本实验实现了快速选择算法，用于在无序数组中查找第 k 小的元素。算法通过分治法利用划分操作，将数组分为左右两部分，每次递归只关注包含第 k 小元素的一部分。其核心思想与快速排序相似，但不同的是快速选择不需要对整个数组进行排序，只通过递归到合适的部分来找到目标元素，从而提高效率。平均情况下，快速选择的时间复杂度为 $O(n)$，空间复杂度为 $O(1)$，但最坏情况下可能退化为 $O(n^2)$。

【5、AC 截图及运行测试截图】

5579613	226001624	1000	**Accepted**	--- KB	--- MS	G++	--- B	2025-03-03 16:06:32



【1、 棋盘覆盖（2-1001）】

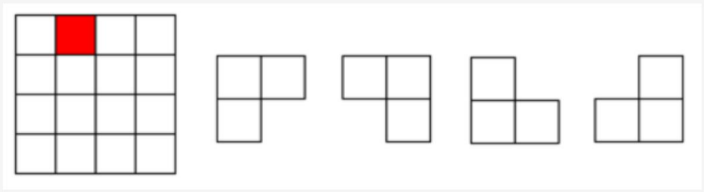
1001 Problem B

时间限制： 2000/1000 MS(Java/Others) | **内存限制：** 65536/32768 KB(Java/Others)

Submits： 89 | **Solved：** 30

题目描述

在一个2^k×2^k 个方格组成的棋盘中，恰有一个方格与其他方格不同，称该方格为一特殊方格，且称该棋盘为一特殊棋盘。在棋盘覆盖问题中，要用图示的4种不同形态的L型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何2个L型骨牌不得重叠覆盖。



【2、 程序代码及注释】

```
/*
输入一个整数 k，k<=5；
输入特殊格子的坐标 x，y

输出一个由数值表示的二维矩阵。填充规则如下：

（1）用数值填充方格；

（2）特殊方格数值为 0；

（3）从中心点开始；然后左上、右上、左下、右下的计数顺序填数；同一块用相同数值表示；
```

（4）每个数值占 4 个位置空间；右对齐，左补空格。

可以使用分治法递归地将一个大棋盘分割成四个小区域，并为每个区域分配编号。递归的终止条件是棋盘大小为 1 时，直接返回。

每次递归判断特殊点所在的子区域，若特殊点不在该区域，则为该区域分配编号，并继续递归其他子区域。

通过这种方式，算法逐步缩小问题规模，直到所有区域都被编号。分治法保证了算法的高效性，并确保最终输出的棋盘布局满足要求。

*/

```cpp
#include<bits/stdc++.h>
#include<cmath>
using namespace std;

// state 数组用于记录棋盘每个位置的编号
int state[100][100];
int cnt = 0;  // 放置编号的计数器

// play 函数实现棋盘的递归分割与编号
// x, y 是特殊点的坐标，a, b 是当前子问题的左上角点坐标，size 是当前棋盘的边长
void play(int x, int y, int a, int b, int size) {
    if (size == 1) {  // 基础条件，当棋盘大小为 1 时结束递归
        return;
    }

    int l = size / 2;  // 子棋盘的边长
    cnt++;  // 增加当前的编号
    int index = cnt;  // 当前编号的值

    // 判断特殊点是否在当前子棋盘的左上角区域
    if (x < a + l && y < b + l) {
        play(x, y, a, b, l);  // 特殊点在左上角区域，递归填充左上角
    } else {
        state[a + l - 1][b + l - 1] = index;  // 否则在右下角区域，放置编号
        play(a + l - 1, b + l - 1, a, b, l);  // 递归填充右下角
    }

    // 判断特殊点是否在左下角区域
    if (x < a + l && y >= b + l) {
        play(x, y, a, b + l, l);  // 特殊点在左下角区域，递归填充左下角
```

```
    } else {
        state[a + l - 1][b + l] = index;  // 否则在右上角区域，放置编号
        play(a + l - 1, b + l, a, b + l, l);  // 递归填充右上角
    }

    // 判断特殊点是否在右上角区域
    if (x >= a + l && y < b + l) {
        play(x, y, a + l, b, l);  // 特殊点在右上角区域，递归填充右上角
    } else {
        state[a + l][b + l - 1] = index;  // 否则在左下角区域，放置编号
        play(a + l, b + l - 1, a + l, b, l);  // 递归填充左下角
    }

    // 判断特殊点是否在右下角区域
    if (x >= a + l && y >= b + l) {
        play(x, y, a + l, b + l, l);  // 特殊点在右下角区域，递归填充右下角
    } else {
        state[a + l][b + l] = index;  // 否则在左上角区域，放置编号
        play(a + l, b + l, a + l, b + l, l);  // 递归填充左上角
    }
}

int main() {
    int k, x, y;
    cin >> k >> x >> y;  // 输入棋盘的层数 k 和特殊点的坐标(x, y)

    int m = pow(2, k);  // 棋盘的边长为 2 的 k 次方

    // 调用 play 函数填充棋盘
    play(x, y, 0, 0, m);

    // 输出棋盘
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < m; j++) {
            printf("%4d", state[i][j]);  // 格式化输出，每个数字占 4 个字符宽度
        }
        cout << endl;  // 每行输出后换行
    }
```

```
69.
70.     return 0;
71.}
```

【3、遇到的问题及解决方法】

Wrong answer，在判断特殊点的位置时要小心和注意，在各个小块中放置骨牌时也需要注意坐标，否则会出错。例如在计算新的特殊点坐标，以及递归给出参数的时候，需要和当前的左上角以及 size 进行相关计算，可以用一个特殊的例子和具体的数据进行确认计算结果是否正确。

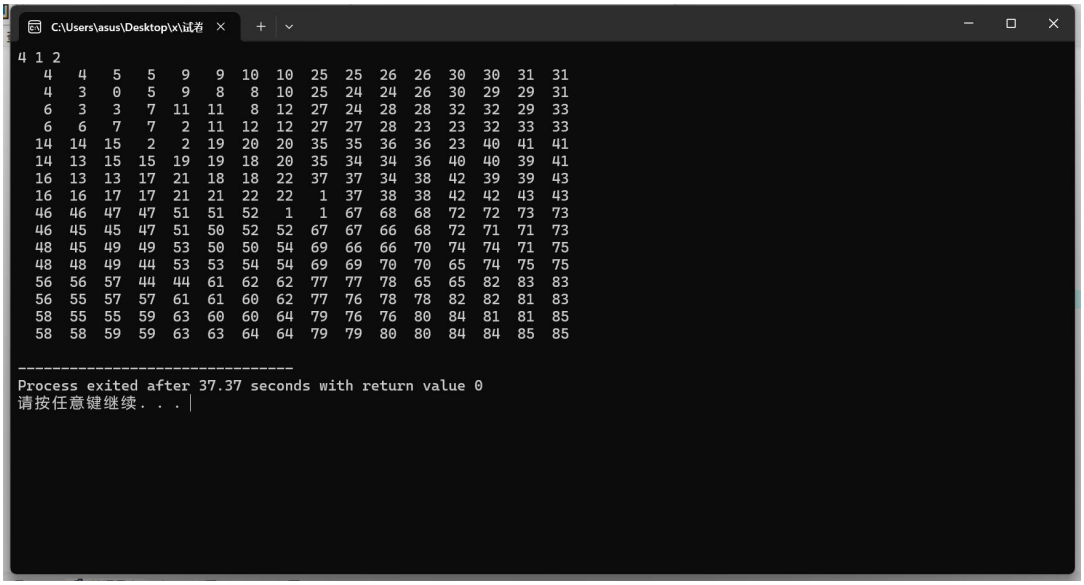
【4、算法分析与知识点】

这个算法的思想是通过分治的方式将二维矩阵分割成多个小正方形区域，并根据特殊格子所在的区域放入对应的骨牌。通过递归地处理子问题，最终完成整个矩阵的填充。

1. 在 `play` 函数中，首先判断边长 `size` 是否为 1，如果是，则直接返回。
2. 然后，增加一个计数器 `cnt`，表示骨牌的索引值。
3. 接下来，通过对特殊格子的坐标与正方形左上角的坐标进行比较，判断特殊格子所在的位置。根据不同的位置，将骨牌放入相应的区域。
   - 如果特殊格子在左上角的区域，不放入骨牌，继续递归处理左上角的子区域。
   - 如果特殊格子在右下角的区域，放入一小块骨牌，并递归处理右下角的子区域。
   - 如果特殊格子在左下角的区域，放入一小块骨牌，并递归处理左下角的子区域。
   - 如果特殊格子在右上角的区域，放入一小块骨牌，并递归处理右上角的子区域。
4. 通过递归调用 `play` 函数，不断分割正方形，并在合适的位置放入骨牌，直到边长为 1 时停止递归。

【5、AC 截图及运行测试截图】

5579454	226001624	1001	**Accepted**	--- KB	--- MS	G++	--- B	2025-03-03 15:36:38



【1、整数分割(1)（2-1002）】

1002 Problem C

时间限制： 2000/1000 MS(Java/Others) | **内存限制：** 131072/65536 KB(Java/Others)

Submits： 141 | **Solved：** 35

题目描述

N年M月O日CX和ZJS同学在刚学玩整数加减后，就在那里比试谁厉害。比试内容是：将一个正整数N拆成若干个正整数的和。看谁拆的多谁就赢。为了赢得比赛，CX就向你求助一个整数的所有拆分方法。

【2、程序代码及注释】

/*

将一个正整数 N 拆成若干个正整数的和。看谁拆的多谁就赢。为了赢得比赛，CX 就向你求助一个整数的所有拆分方法。

输入 N（ 1 <= N <= 50 ）。

对于 N 的拆分方法，请以字典序排列（数字越大越排前面）。每一种拆分之间以一个空格分开。

可以通过深度优先搜索（DFS）来找到所有符合条件的整数分拆组合。目标是将数字 n 分解为一系列数字之和，并且这些数字按照非递减顺序排列。

首先，定义了 nums 数组存储当前组合，size1 记录组合的大小。通过递归从 n 开始，逐步尝试将数字添加到当前组合中，直到和为 n。为了保证数字按非递减顺序，递归时限制当前数字不小于上一个数字。

当和为 n 时，输出当前组合。每次递归时，条件 if(i <= last && i + sum <= n) 保证了数字顺序和组合和的正确性。最终，输出所有符合条件的分拆组合。

*/

```
#include<iostream>
using namespace std;

int n; // 输入的目标数字
int nums[50] = {0}; // 存储当前的数字组合
int size1=0; // 记录当前组合的大小，用于管理 nums 数组的索引

// 深度优先搜索函数，cur 表示当前待拆分的数字，sum 表示当前组合的和，last 限制当前数字的大小
void dfs(int cur, int sum, int last){
    if(cur == 0){  // 当 cur 为 0 时，表示已经找到一个有效的组合
        // 输出当前组合
        for(int i = 0; i < size1 - 1; i++){
            cout << nums[i] << " ";  // 输出组合中的每个数字
        }
        cout << nums[size1 - 1] << endl;  // 输出组合中的最后一个数字
    }
    else {
        for(int i = cur; i >= 1; i--){  // 从 cur 到 1 遍历数字，
```

```
    保证组合中的数字按非递减顺序
            // 判断当前数字是否符合条件：i <= last 且 sum + i <= n
            if(i <= last && i + sum <= n){
                nums[size1] = i;  // 将当前数字加入组合
                size1++;  // 增加组合的大小
                dfs(cur - i, sum + i, i);  // 递归搜索剩余的数字
                size1--;  // 回溯，撤销当前选择
            }
        }
    }
}

int main(){
    // 不断读取输入的 n，直到没有更多输入
    while(cin >> n){
        size1 = 0;  // 重置组合大小
        dfs(n, 0, n);  // 调用深度优先搜索，从 n 开始搜索
    }
}
```

【3、遇到的问题及解决方法】

Wrong answer，OJ 中要求实现多次输入，可采用 While(cin>>n)来实现多次输入.

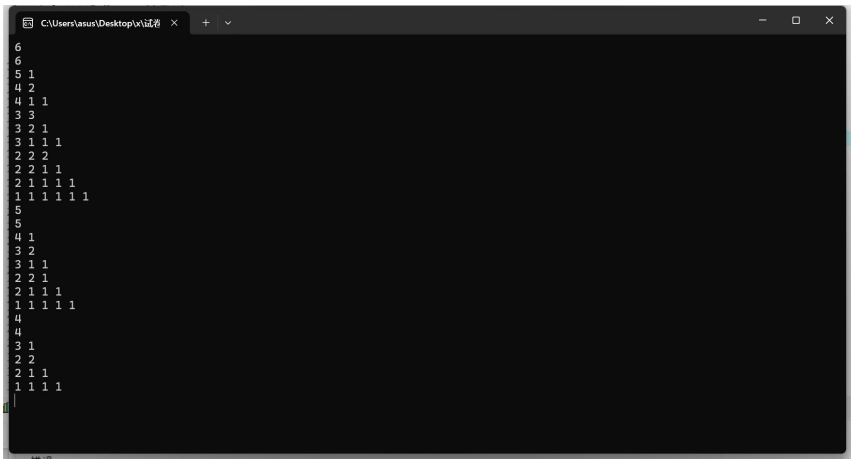
Presentation Error,每一个方案的最后一个换行不能添加空格。

【4、算法分析与知识点】

本实验使用了深度优先搜索算法（DFS），实现了将一个正整数 N 拆分成若干个正整数的和，并按照字典序排列从大到小输出所有的拆分方法。在 DFS 算法中，定义了一个递归函数 dfs，其中 cur 表示当前需要分解的数，sum 表示已经分解的数的和，last 表示上一个加数。在递归实现中，首先对当前节点进行处理，然后递归地访问它的相邻节点，直到遇到终止条件才返回。

【5、AC 截图及运行测试截图】

5579447	226001624	1002	**Accepted**	--- KB	--- MS	G++	--- B	2025-03-03 15:34:29



Module 3: 递推/递归与分治策略堂练

1000 Problem A

时间限制： 2000/1000 MS(Java/Others) | **内存限制：** 65536/32768 KB(Java/Others)

Submits： 104 | **Solved：** 7

题目描述

组合学中有这样一个问题：某人给五个朋友写信，邀请他们来家中聚会。请柬和信封交由助手去处理。粗心的助手却把请柬全装错了信封。请问：助手会有多少种装错的可能呢？

这个问题是是由当时有名的数学家约翰·伯努利(Johann Bernoulli，1667—1748)的儿子

丹尼尔·伯努利(Danid Bernoulli，1700—1782)提出来的。

```cpp
#include<iostream>
using namespace std;

// 递归函数 Count
// 功能：计算 n 个元素的错位排列（Derangement）的数量
// 错位排列：排列中没有任何一个元素出现在其原始位置
// 参数：n 表示元素的数量
// 返回值：n 个元素的错位排列的数量
int Count(int n) {
    // 递归终止条件
    if (n == 1)  // 当 n = 1 时，错位排列的数量为 0（因为唯一一个元素只能在其原始位置）
        return 0;
    if (n == 2)  // 当 n = 2 时，错位排列的数量为 1（两个元素交换位置）
        return 1;
    else
        // 递归公式：D(n) = (n-1) * (D(n-1) + D(n-2))
        // D(n-1) 表示第 n 个元素与前面某个元素交换位置后的错位排列数
        // D(n-2) 表示第 n 个元素与前面某个元素交换位置后，另一个元素也交换位置的错位排列数
        return (n - 1) * (Count(n - 1) + Count(n - 2));
}
```

```
21.
22.int main() {
23.    int n;
24.    // 循环读取输入的 n
25.    while (cin >> n) {
26.        // 调用 Count 函数计算错位排列的数量并输出
27.        cout << Count(n) << endl;
28.    }
29.    return 0;
30.}
```

1001 Problem B

时间限制：2000/1000 MS(Java/Others) | 内存限制：65536/32768 KB(Java/Others)

Submits：86 | Solved：4

题目描述

一个大学生按在校4年整计算，假设每个月的生活费需要x元整，银行年利率为rate，该生在每个月计息后取当月生活费，那么如果父母在第一个月初需要存入多少钱（后续不再存），才能使得在最后一个月刚好取完？请使用递推方式求解。

输入要求

一个整数x表示每月的生活费；一个浮点数表示银行的年利率。

输出要求

一个浮点数表示第一个月需要存入多少钱，保留小数点后2位。

```
1.  #include <iostream>
2. #include <cmath>
3. #include <iomanip>
4.
5. using namespace std;
6.
7. /**
8.  * 计算第一个月初需要存入的金额，使得在未来 48 个月每月支取 x 元，账户余额刚好用完
9.  * @param x 每月需要支取的生活费
10. * @param rate 年利率（小数形式，例如 5% 输入为 0.05）
11. * @return 初始存款金额
12. */
13.double cal(double x, double rate) {
14.    double r = rate / 12.0; // 计算月利率（年利率除以 12）
15.    double balance = 0.0;   // 假设最后一个月取完钱后余额刚好为 0
16.
17.    // 逆向递推计算第一个月初的存款
```

```
    for (int i = 0; i < 48; ++i) { // 逆向遍历 48 个月
        balance = (balance + x) / (1 + r); // 反推前一个月的存款余额
    }

    return balance; // 返回第一个月初的存款金额
}

int main() {
    double x, rate;

    // 输入每月生活费 x 和年利率 rate
    cin >> x;
    cin >> rate;

    // 计算第一个月初需要存入的金额
    double initialDeposit = cal(x, rate);

    // 输出计算结果，保留两位小数
    cout << fixed << setprecision(2) << initialDeposit << endl;

    return 0;
}
```

1002 Problem C

时间限制： 2000/1000 MS(Java/Others) | **内存限制：** 65536/32768 KB(Java/Others)

Submits： 76 | **Solved：** 0

题目描述

给定一个自然数n，由n开始可以依次产生半数集set(n)中的数如下。
(1) n∈set(n)；
(2) 在 n的左边加上一个自然数，但该自然数不能超过最近添加的数的一半；
(3) 按此规则进行处理，直到不能再添加自然数为止。
例如，set(6)={6,16,26,126,36,136}。半数集 set(6)中有 6 个元素。
注意半数集是多重集。

对于给定的自然数n，编程计算半数集set(n)中的元素个数。

```
#include <iostream>
#include <fstream>
using namespace std;

/**
 * 递归计算半数集 set(n) 的大小
 * @param n 当前数值
 * @param last 最近添加的数字（用于限制可添加的范围）
```

```
 * @return 半数集 set(n) 的元素个数
 */
int countSet(int n, int last) {
    int count = 1; // 计算自身（当前 n 本身是一个有效的集合）

    // 遍历可以添加的前缀数字 i
    for (int i = 1; i <= last / 2; ++i) { // 只能添加 last/2 以内的数字
        count += countSet(i * 10 + n, i); // 递归添加 i 作为前缀
    }

    return count; // 返回当前 n 对应的半数集元素个数
}

int main() {
    ifstream input("input.txt");  // 读取 input.txt 文件
    ofstream output("output.txt"); // 写入 output.txt 文件

    int n;
    input >> n; // 读取输入的自然数 n

    int result = countSet(n, n); // 计算 set(n) 的大小

    output << result << endl; // 将结果写入 output.txt 文件

    return 0;
}
```

1003 Problem D

时间限制：2000/1000 MS(Java/Others) | 内存限制：65536/32768 KB(Java/Others)

Submits：64 | Solved：0

题目描述

n 个元素的集合{
,
2
,
,
n
1 ⊡ }可以划分为若干个非空子集。例如，当n=4时，集合{1，2，
3，4}可以划分为15个不同的非空子集如下：
{{1}，{2}，{3}，{4}}，
{{1，2}，{3}，{4}}，
{{1，3}，{2}，{4}}，
{{1，4}，{2}，{3}}，
{{2，3}，{1}，{4}}，
{{2，4}，{1}，{3}}，
{{3，4}，{1}，{2}}，
{{1，2}，{3，4}}，
{{1，3}，{2，4}}，
{{1，4}，{2，3}}，
{{1，2，3}，{4}}，
{{1，2，4}，{3}}，
{{1，3，4}，{2}}，
{{2，3，4}，{1}}，
{{1，2，3，4}}

给定正整数n，计算出n个元素的集合{
,
2
,
,
n
1 ⊡ }可以划分为多少个不同的非空子集。

```cpp
#include <iostream>
#include <fstream>
#include <vector>
using namespace std;

/**
 * 计算 n 元素集合的不同非空子集划分数
 * 使用动态规划方法计算 Bell 数的递推关系
 * @param n 给定的集合大小
 * @return 集合 {1,2,...,n} 的划分方式总数
 */
int calcu(int n) {
    // 创建一个 (n+1) x (n+1) 的动态规划表 N，初始化为 0
    vector<vector<int>> N(n + 1, vector<int>(n + 1, 0));

    // Bell 数的初始条件：N[0][0] = 1
    N[0][0] = 1;

    // 计算 Bell 三角形表
    for (int i = 1; i <= n; i++) {
        // 每一行的第一个元素 N[i][0] 等于上一行的最后一个元素 N[i-1][i-1]
        N[i][0] = N[i - 1][i - 1];

        // 计算当前行的其余元素
```

```
        for (int j = 1; j <= i; j++) {
            N[i][j] = N[i - 1][j - 1] + N[i][j - 1];
        }
    }

    // 返回 Bell 数，即 N[n][0]
    return N[n][0];
}

int main() {
    ifstream input("input.txt");  // 读取 input.txt 文件
    ofstream output("output.txt"); // 写入 output.txt 文件

    int n;
    input >> n; // 读取输入的整数 n

    int result = calcu(n); // 计算 Bell 数（即 n 的划分数）

    output << result << endl; // 将计算结果写入 output.txt

    return 0;
}
```

【第二部分】

【同类题型实验扩展/可选/会影响分值】

1、题目+测试数据+结果

2、分析

3、建议：难度不低于实验题

【1、求逆序对】

求逆序对个数

题目描述

有一实数序列A[1]、A[2]、A[3]、……A[n-1]、A[n] (n<10000)，若i<j，并且A[i]>A[j]，则称A[i]与A[j]构成了一个逆序对，求数列A中逆序对的个数。
例如，5 2 4 6 2 3 2 6，可以组成的逆序对有
(5 2)，(5 4)，(5 2)，(5 3)，(5 2)，
(4 2)，(4 3)，(4 2)，
(6 2)，(6 3)，(6 2)，
(3 2)
共：12个

输入
共两行，第一行有一个正整数n，第二行有n个整数。
输出
只有一行为逆序对个数。

样例输入 Copy
8
5 2 4 6 2 3 2 6
样例输出 Copy
12

【2、程序代码及注释】

/*给定一个序列 a1,a2,⋯,an，如果存在 i<j 并且 ai>aj，那么我们称之为逆序对，求逆序对的数目。

要求用分治法求解。在拆分阶段，数组被递归地划分为两个子数组，直到每个子数组仅包含一个元素（自然是有序的）。在合并阶段，左右两个子数组逐步合并成一个有序数组，并在此过程中统计逆序对的数量。由于左半部分和右半部分分别是有序的，因此当发现左半部分的某个元素大于右半部分的元素时，可以断定左半部分当前元素及其后续所有元素都与该右半部分元素构成逆序对。

*/

```
1. #include<iostream>
2. using namespace std;
3. static int cnt = 0;
4. int a[10000] = {0};
5. static void merge(int array[],int p,int q,int r) //分治法中合并两块，p,q,r 分别为起始、中间、结束索引
6. {
7.  int k = 0;
8.  int begin1,end1,begin2,end2; //分别代表左右两块的起始/终止索引
9.  int* temp = new int[r-p+1];  //创建临时数组 temp
10. begin1 = p; end1 = q;
11. begin2 = q+1; end2 = r;
12. while((begin1 <= end1)&&(begin2 <= end2)){ //判断两边都未排序完成
```

```
13.   if(array[begin1]<=array[begin2]){
14.    temp[k] = array[begin1];
15.    begin1++;
16.   }
17.   else{       //左边的序列要大于右边，存在逆序对，且因为两块是排列完的序列，所以左边接下来的数列都要大。
18.    temp[k] = array[begin2];
19.    begin2++;
20.    cnt += end1 - begin1 + 1;
21.   }
22.   k++;
23. }
24. //将剩余的数加入临时数组 temp
25. while(begin1<=end1){
26.   temp[k++] = array[begin1++];
27. }
28. while(begin2<=end2){
29.   temp[k++] = array[begin2++];
30. }
31. //将临时数组存入数组中。
32. for(int i=0;i<(r-p+1);i++){
33.   array[p+i] = temp[i];
34. }
35. delete[] (temp);
36.}
37.
38.//分治排序
39.void MergeSort(int array[],int p,int r){
40. if(p<r){
41.   int Mid = (p+r)/2;
42.   MergeSort(array,p,Mid);
43.   MergeSort(array,Mid+1,r);
44.   merge(array,p,Mid,r);
45. }
46.}
47.
48.int main(){
49. int n;
50. cin>>n;
51. for(int i=0;i<n;i++)
52.   cin>>a[i];
53. MergeSort(a,0,n-1);
54. cout<<cnt<<endl;
```

```
55.  return 0;
56.}
```

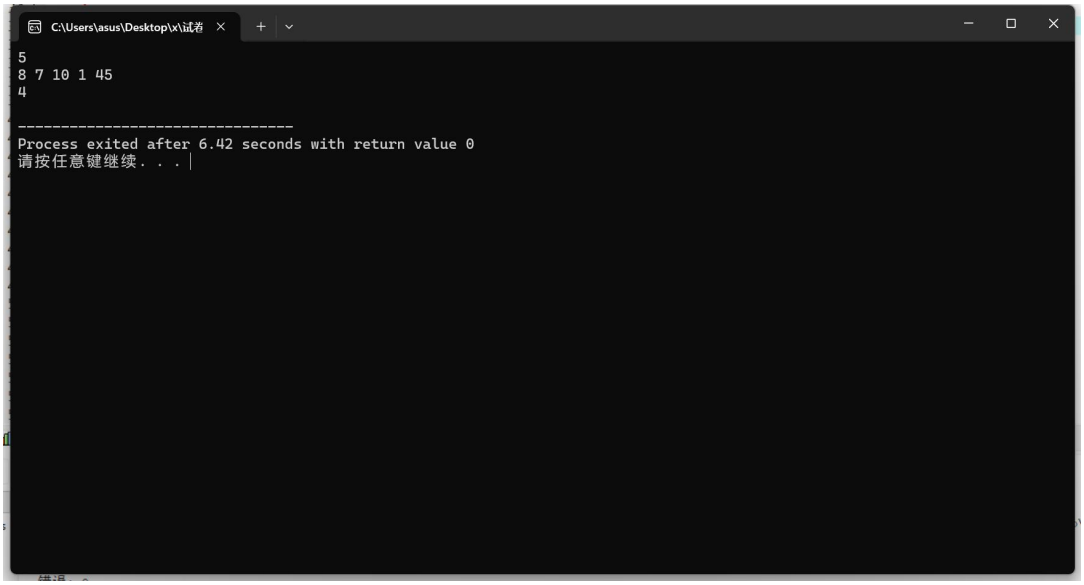
【3、遇到的问题及解决方法】

Wrong Answer，首先逆序对要前者大于后者才行，所以 if(array[begin1]<=array[begin2]) 在判断时要使用小于等于号。其次，在发现逆序对后，由于左右两侧都是已经排列好的从小到大的数组，所以说明左边剩余的数与右边该数均为逆序对，故 cnt 加的数为左边数组剩余长度 cnt += end1 - begin1 + 1。

【4、算法分析与知识点】

分治算法的思想是通过分治的方式将原始序列不断划分为更小的子序列，然后对子序列进行排序，并在合并的过程中计算逆序对的数目。通过递归地处理子问题，最终得到整个序列的逆序对数目。

【5、AC 截图及运行测试截图】



【1、双色 Hanoi 塔】

【2、程序代码及注释】

```
/*
```

设 A、B、C 是 3 个塔座。开始时，在塔座 A 上有一叠共 n 个圆盘，这些圆盘自下而上，由大到小地叠在一起。各圆盘从小到大编号为 1，2，……，n，奇数号圆盘着蓝色，偶数号圆盘着红色，如图所示。现要求将塔座 A 上的这一叠圆盘移到塔座 B 上，并仍按同样顺序叠置。在移动圆盘时应遵守以下移动规则：

规则(1)：每次只能移动 1 个圆盘；

规则(2)：任何时刻都不允许将较大的圆盘压在较小的圆盘之上；

规则(3)：任何时刻都不允许将同色圆盘叠在一起；

规则(4)：在满足移动规则(1)-(3)的前提下，可将圆盘移至 A，B，C 中任一塔座上。

给定的正整数 n

```
*/
```

```cpp
#include<iostream>
using namespace std;

// 移动函数
// 功能：输出移动步骤
// 参数：
//    n: 当前移动的盘子编号
//    scr: 移动的起始位置
//    dst: 移动的目标位置
void move(int n, char scr, char dst) {
    cout << n << " " << scr << " " << dst << endl;  // 输出移动步骤
}

// 汉诺塔递归函数
// 功能：解决汉诺塔问题，输出移动步骤
// 参数：
//    n: 当前需要移动的盘子数量
//    src: 起始位置
//    dst: 目标位置
//    mid: 中间辅助位置
void hanoi(int n, char src, char dst, char mid) {
    // 递归终止条件：如果只有一个盘子，直接移动到目标位置
    if (n == 1) {
        move(1, src, dst);  // 将盘子 1 从 src 移动到 dst
    }
    else {
        // 将前 n-1 个盘子从 src 移动到 mid，借助 dst 作为辅助位置
        hanoi(n - 1, src, mid, dst);

        // 将第 n 个盘子从 src 移动到 dst
        move(n, src, dst);

        // 将前 n-1 个盘子从 mid 移动到 dst，借助 src 作为辅助位置
        hanoi(n - 1, mid, dst, src);
    }
}

int main() {
    int n;
    cin >> n;  // 输入盘子的数量
    hanoi(n, 'A', 'B', 'C');  // 调用汉诺塔函数，起始位置为 'A'，目
```

```
    标位置为 'B'，中间位置为 'C'
    return 0;
}
```

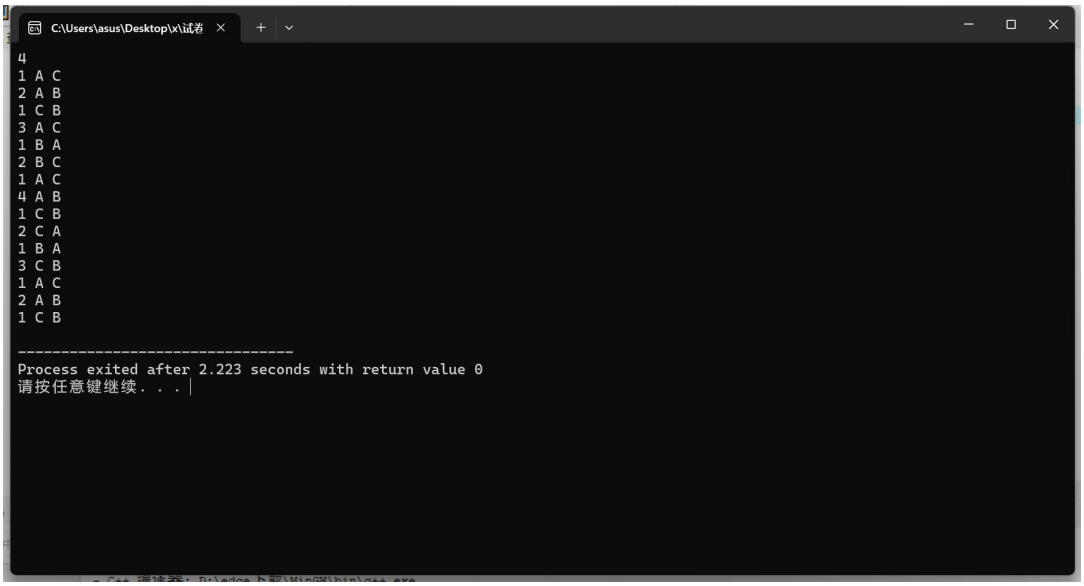
【3、遇到的问题及解决方法】

注意题目中 B 才是目标位置，在调用函数时要注意参数的输入。

【4、算法分析与知识点】

将大象放进冰箱要三步，打开冰箱门，把大象放进冰箱，关上冰箱门。同理汉诺塔游戏也可以分为这样三步，若我们要把 n 片移动至目标位置，则说明 n-1 片也是可以移动的，那么我们只需要先将 n-1 片移动至中间位置，再将第 n 片从源位置移动至目标位置，接着再将 n-1 片从中间位置移动至目标位置即可。但要注意一个问题，如此做的前提是 n=1 时可行，显然这是合理的，当 n=1 时，我们只需直接将圆盘移动至目标位置即可。这似乎与高中学的数学归纳法有些类似。

【5、AC 截图及运行测试截图】



【1、日程安排（3-1002）】

【2、程序代码及注释】

```
/*
设有 n=2^k 个运动员要进行网球循环赛。现要设计一个满足以下要求的比赛日程表：

(1)每个选手必须与其他 n-1 个选手各赛一次；

(2)每个选手一天只能参赛一次；

(3)循环赛在 n-1 天内结束。


#include<iostream>
```

```cpp
#include <cmath>
using namespace std;

int a[100][100];  // 定义一个全局二维数组，用于存储矩阵

// 非递归函数 fun1
// 功能：填充一个 2^k x 2^k 的矩阵
// 参数：k 表示矩阵的大小为 2^k x 2^k
void fun1(int k) {
    int n = 2;  // 初始矩阵大小为 2x2
    int temp;   // 用于保存当前矩阵大小
    int i, j;   // 循环变量

    // 初始化 2x2 的矩阵
    a[1][1] = 1;
    a[1][2] = 2;
    a[2][1] = 2;
    a[2][2] = 1;

    if (k == 1) {
        return;  // 当 k == 1 时，直接返回
    } else {
        // 循环扩展矩阵大小，从 2x2 扩展到 2^k x 2^k
        for (int t = 1; t < k; t++) {
            temp = n;  // 保存当前矩阵大小
            n *= 2;    // 扩展矩阵大小为原来的 2 倍

            // 填充左下角
            for (i = temp + 1; i <= n; i++) {
                for (j = 1; j <= n / 2; j++) {
                    a[i][j] = a[i - temp][j] + temp;  // 左下角的值等于左上角的值加上 temp
                }
            }

            // 填充右上角
            for (i = 1; i <= n / 2; i++) {
                for (j = temp + 1; j <= n; j++) {
                    a[i][j] = a[i + temp][j - temp];  // 右上角的值等于左下角的值
                }
            }

```

```
            // 填充右下角
            for (i = temp + 1; i <= n; i++) {
                for (j = temp + 1; j <= n; j++) {
                    a[i][j] = a[i - temp][j - temp];  // 右下角的值等于左上角的值
                }
            }
        }
    }
}

// 输出矩阵
// 参数：n 表示矩阵的大小为 n x n
void show(int n) {
    int i, j;  // 循环变量
    for (i = 1; i <= n; i++) {
        for (j = 1; j < n; j++) {
            cout << a[i][j] << " ";  // 输出矩阵的每个元素，用空格分隔
        }
        cout << a[i][j] << endl;  // 输出每行的最后一个元素并换行
    }
}

int main() {
    int n, k;
    cin >> k;  // 输入 k，表示矩阵的大小为 2^k x 2^k
    n = pow(2, k);  // 计算矩阵的大小 n = 2^k
    fun1(k);  // 调用 fun1 函数填充矩阵
    show(n);  // 调用 show 函数输出矩阵
    return 0;
}
```

【3、遇到的问题及解决方法】

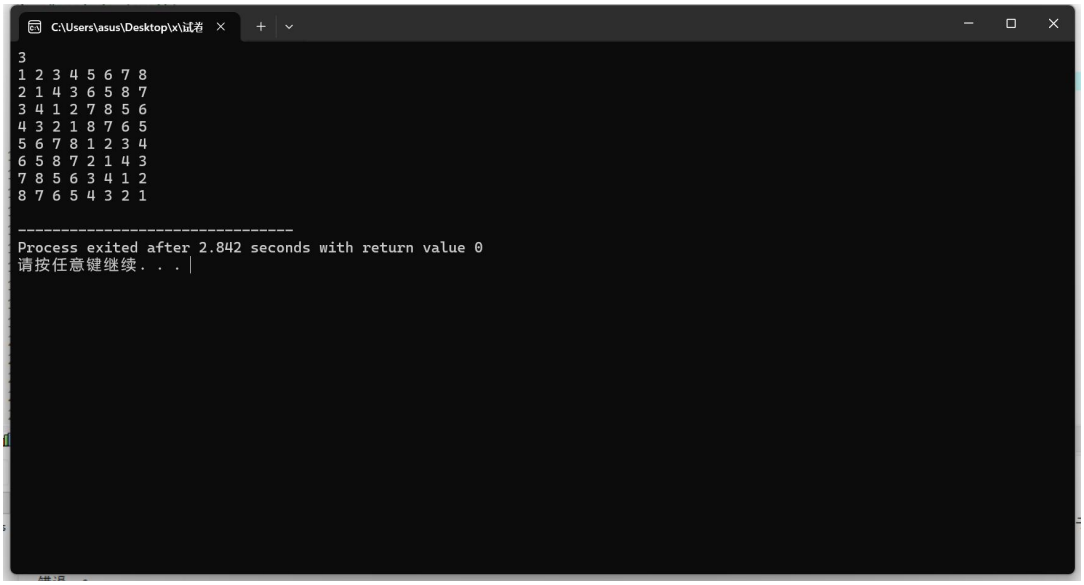
PE，在输出每行的最后一个字符时不能带空格，要直接换行，所以在第二层循环进行输出。

【4、算法分析与知识点】

在方法 1 中，我们采用了四层的循环嵌套，最外层循环表示我们当前位于第几层大循环，n 表示共有将列数分为几部分，t 表示位于第几层子循环，其由 n 进行控制，i, j 分别表示当前选手编号和比赛天数，其中循环开始的初始值为每个子正方形的右下角的坐标，接着我们再由 for 循环进行赋值，右下角中正方形的值等于左上角中正方形的值，左下角正方形的值右上角正方形的值。在方法二中我们首先自底向上，如果时两个运动员进行比赛，那么结果非常容易产生。那如果是四个运动员呢？我们依旧可以把它先进

行分解。得出左上角的运动员赛程，然后为了不重复且比完，我们可以将左上角的赛程填入右下角中，再进行规律查找，我们发现左下角和右上角的运动员编号可以由左上角运动员编号的值加上分割正方形的边长得到，由此我们得到了四种分治情况，接下来只需要进行递归，指导正方形的边长为一后进行返回即可。

【5、AC 截图及运行测试截图】



【1、小明的烦恼】

小明的烦恼详解（C++）

技术标签：C++

题目描述:

小明最近新买了一个房间，为了给它做装修，想要给它铺上地砖。然而现有的地砖只有两种规格分别为1米*1米、2米*2米，由于小明买的房间有点小，宽度只有3米，长度为N米。当然这样一个房间也足够他自己一个人住了。那么如果要给这个房间铺设地砖，且只用以上这两种规格的地砖，请问有几种铺设方案。

【2、程序代码及注释】

/*

核心思想是大数处理与递推计算。由于普通的整型数据类型（如 int 或 long long）无法存储非常大的数，因此采用了高精度计算的方法，即用二维数组来存储大数的分段值，每个数被拆分成最多 100 个 5 位数 的段，从而支持超大数的运算。

*/

```
#include<iostream>
using namespace std;

// 用于计算的函数
// 功能：计算并输出一个特定数列的第 n 项
// 参数：n 表示需要计算的项数
void fun(int n) {
    int p, q;
```

```
    int s = 0;  // 用于检测是否为第一个数
    int a[1000][100] = {0};  // 使用二维数组来解决大数问题，每行存储一个数的 100 个五位数字段

    // 起始状态，初始化前三个数
    a[0][99] = 1;  // 第一个数的最后五位是 1
    a[1][99] = 3;  // 第二个数的最后五位是 3
    a[2][99] = 5;  // 第三个数的最后五位是 5

    int m;  // 用于临时存储计算结果
    for (p = 3; p < n; p++) {
        // 计算当前项的值：a[p] = 2 * a[p-2] + a[p-1]
        m = 2 * a[p - 2][99] + a[p - 1][99];  // 先计算最后五位
        a[p][99] = m % 100000;  // 取余 100000，得到当前项的最后五位

        // 从倒数第二位开始向前计算
        for (q = 98; q >= 0; q--) {
            // 计算第 q 位的值：2 * a[p-2][q] + a[p-1][q] + 进位
            m = 2 * a[p - 2][q] + a[p - 1][q] + m / 100000;  // 加上上一位的进位
            a[p][q] = m % 100000;  // 取余 100000，得到当前项的第 q 位
        }
    }

    // 输出第 n 项的结果
    for (int j = 0; j < 100; j++) {
        if (a[n - 1][j] != 0 && s == 0) {
            // 第一个非零的五位数字段，直接输出（不补零）
            cout << a[n - 1][j];
            s = 1;  // 标记已经输出了第一个非零字段
        } else if (s == 1) {
            // 其他五位数字段，按照五位固定输出（不足五位补零）
            printf("%05d", a[n - 1][j]);
        }
    }
    cout << endl;  // 输出换行
}

int main() {
    int C;  // 测试用例的数量
    int n;  // 需要计算的项数
```

```
    cin >> C;  // 输入测试用例的数量

    // 循环处理每个测试用例
    for (int i = 0; i < C; i++) {
        cin >> n;  // 输入需要计算的项数
        fun(n);    // 调用 fun 函数计算并输出结果
    }

    return 0;
}
```

【3、遇到的问题及解决方法】

注意题目中大数问题的处理。在本题中为了实现对大数问题的处理，我们定义了一个二维数组，数组的每一个单元存放五位数字来防止溢出，最后再将存储的数字连接起来打印输出。

【4、算法分析与知识点】

本实验使用了动态规划来解决问题，首先我们将原问题分解成若干子问题，若将长度为 N 的总长度分拣成 1 米和 N-1 米，则 1 米长的地砖只有一种铺法，接着我们再将长度为 N 的总长度分拣成 2 米和 N-2 米，得到三种铺法，但由于有一种铺法与第一种重复，所以可以看作两种。由此我们可以得到计算公式 f[n]=f[n-1]+2*f[n-2]，接着再初始化边界条件，得到铺砖列数分别为 1、2、3 时的不同方法为 1，3，5，最后使用递推计算得到结果。

【5、AC 截图及运行测试截图】

